

AMD HIP 实验 Notebook 填空报告

1 Lab 1: Vector Addition

1.1 线程索引计算

一维网格中的全局线程编号为

$$\text{globalIdx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

1.2 Vector Add Kernel 填空

```
1 __global__ void vector_add(const float* A, const float* B, float* C,  
2   int N) {  
3     int idx = blockIdx.x * blockDim.x + threadIdx.x;  
4  
5     if (idx < N) {  
6         C[idx] = A[idx] + B[idx];  
7     }  
}
```

1.3 启动配置计算

当 $N = 1000$ 时:

Block Size	Grid Size	Total Threads	Idle Threads	Utilization
64	16	1024	24	97.66%
128	8	1024	24	97.66%
256	4	1024	24	97.66%
512	2	1024	24	97.66%

这四种 block size 在该例中的线程利用率相同。实际性能不一定完全由线程利用率决定, 还会受到 wavefront 对齐、occupancy、寄存器使用、内存带宽和调度开销影响。Vector Add 的计算量很小, 主要瓶颈是全局内存读写带宽, 因此不同 block size 的性能差异通常不会像计算密集型 kernel 那样明显。

1.4 Wavefront 分析

默认 wave32 模式下, block size 为 100 时:

$$\left\lceil \frac{100}{32} \right\rceil = 4$$

因此每个 block 需要 4 个 wavefront, 浪费 lane 数为

$$4 \times 32 - 100 = 28.$$

若使用 wave64:

$$\left\lceil \frac{100}{64} \right\rceil = 2, \quad 2 \times 64 - 100 = 28.$$

即需要 2 个 wavefront, 同样浪费 28 个 lane。

当同一个 wavefront 内线程进入不同 if-else 分支时, 会发生分支发散。硬件通常需要串行执行不同分支路径, 并屏蔽不属于当前路径的 lane, 因此有效并行度下降。

1.5 Occupancy 分析

occupancy 表示一个 CU 上活跃 wavefront 数与最大可支持 wavefront 数的比例。较高 occupancy 有助于隐藏内存延迟, 但并不保证性能最优。若 kernel 受内存带宽、atomic contention 或指令依赖限制, 继续增加 occupancy 可能无法带来明显收益。

1.6 实测性能分析

在 $N = 1,048,576$ 、10 次运行下, 实测结果如下:

Block Size	Mean (ms)	Std (ms)	Min (ms)	Max (ms)
16	0.0355	0.0009	0.0329	0.0359
32	0.0201	0.0009	0.0187	0.0207
64	0.0193	0.0015	0.0177	0.0223
128	0.0202	0.0015	0.0176	0.0226
256	0.0208	0.0018	0.0177	0.0252
512	0.0207	0.0000	0.0206	0.0208
1024	0.0212	0.0014	0.0207	0.0253

最低平均执行时间由 block size 64 取得，为 0.0193 ms。Vector Add 每个元素读取两个 float 并写入一个 float，数据量约为 $3N \times 4 = 12,582,912$ bytes，因此 block size 64 的估算带宽为

$$\frac{12,582,912}{0.0193 \times 10^{-3}} \approx 651.96 \text{ GB/s.}$$

由于带宽和执行时间成反比，本组数据中最高估算带宽同样对应 block size 64。除 block size 16 明显较慢外，32 到 1024 的误差范围有重叠，差异不宜解释为强统计显著。Vector Add 性能主要由内存带宽决定，因为每个元素只有一次加法，计算量远小于内存读写量。

1.7 Sub-wavefront Block Size

sub-wavefront 实测结果如下：

Block Size	Mean (ms)	Std (ms)	Sub-Wave?
8	0.0551	0.0015	YES
16	0.0350	0.0035	YES
32	0.0211	0.0014	no
64	0.0217	0.0049	no
128	0.0206	0.0001	no
256	0.0206	0.0001	no

block size 8 和 16 小于 wave32，会造成一个 wavefront 中只有部分 lane 有效工作。实测中 8 和 16 的平均时间分别为 0.0551 ms 和 0.0350 ms，明显慢于 wave-aligned 的 32、128、256，因此存在明显 sub-wavefront 性能损失。若换成 MI100 的 wave64，则所有小于 64 的 block size 都属于 sub-wavefront，例如 8、16、32。

1.8 多元素每线程 Kernel

```
1 __global__ void vector_add_multi(const float* A, const float* B, float
    * C,
```

```

2                                     int N, int elements_per_thread) {
3     int tid = blockIdx.x * blockDim.x + threadIdx.x;
4     int stride = blockDim.x * gridDim.x;
5
6     for (int i = tid; i < N; i += stride) {
7         C[i] = A[i] + B[i];
8     }
9 }

```

相比一线程处理一个元素，grid-stride loop 能让固定数量的线程覆盖任意大的输入，减少启动过多 block 的需求，并在大规模数据上提高每个线程的工作量和调度效率。

2 Lab 2: Matrix Transpose

2.1 二维索引

二维索引计算为

$$idx = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \quad idy = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}.$$

当 $\text{blockDim}=(32,32)$ 时：

blockIdx	threadIdx	Global (idx, idy)
(0,0)	(5,10)	(5,10)
(1,0)	(0,0)	(32,0)
(2,3)	(15,20)	(79,116)

对于 $\text{rows}=100$, $\text{cols}=200$ 且 block size 为 32×32 的矩阵：

$$\text{blocks.x} = \left\lceil \frac{200}{32} \right\rceil = 7, \quad \text{blocks.y} = \left\lceil \frac{100}{32} \right\rceil = 4.$$

因此总 block 数为 $7 \times 4 = 28$ 。

2.2 转置索引变换

对于 4×6 矩阵：

Thread (idx, idy)	Input Index	Output Index
(0,0)	$0 \times 6 + 0 = 0$	$0 \times 4 + 0 = 0$
(1,0)	$0 \times 6 + 1 = 1$	$1 \times 4 + 0 = 4$
(0,1)	$1 \times 6 + 0 = 6$	$0 \times 4 + 1 = 1$
(3,2)	$2 \times 6 + 3 = 15$	$3 \times 4 + 2 = 14$

写入索引使用 $\text{idx} * \text{rows} + \text{idy}$ ，是因为转置后输出矩阵尺寸为 $\text{cols} \times \text{rows}$ ，输出矩阵每一行包含 rows 个元素，而不是原矩阵的 cols 个元素。

2.3 Naive Transpose Kernel

```

1  __global__ void matrix_transpose_kernel(const float* input, float*
    output,
2                                     int rows, int cols) {
3      int idx = blockIdx.x * blockDim.x + threadIdx.x;
4      int idy = blockIdx.y * blockDim.y + threadIdx.y;
5
6      if (idx < cols && idy < rows) {
7          output[idx * rows + idy] = input[idy * cols + idx];
8      }
9  }
```

2.4 Coalescing 分析

读取 $\text{input}[\text{idy} * \text{cols} + \text{idx}]$ 时，同一行内连续的 threadIdx.x 会访问连续地址，因此读取是 coalesced 的。

写入 $\text{output}[\text{idx} * \text{rows} + \text{idy}]$ 时，连续 threadIdx.x 的写入地址相差 rows 个 float。对于 1024×1024 矩阵，stride 为 1024 个 float，即 4096 bytes。

若一次内存 transaction 获取 128 bytes，也就是 32 个 float，则一个 wave32 做这种非连续写入时，理想情况下每个线程可能落在不同 transaction 中，因此约需要 32 次 transaction。

2.5 Tiled Transpose 分析

共享内存 tile 定义为 $\text{tile}[\text{BLOCK_SIZE}][\text{BLOCK_SIZE} + 1]$ ，其中 +1 用于打破列访问时的 bank conflict，使线程访问共享内存列时不会集中到同一个 bank。

写回时交换 blockIdx.x 和 blockIdx.y ，是因为 tile 需要被转置到输出矩阵的对称位置，从而让读和写都尽量保持连续访问。

当 $\text{BLOCK_SIZE}=32$ 时，共享内存使用量为

$$32 \times (32 + 1) \times 4 = 4224 \text{ bytes.}$$

2.6 实测性能分析

矩阵转置的实测 kernel 时间如下：

Test	Dimensions	Elements	Mean (ms)	Std (ms)
1	16×16	256	0.0078	0.0001
2	128×32	4096	0.0090	0.0000
3	1×1024	1024	0.0076	0.0010
4	1001×2001	2,003,001	0.1117	0.0002

Test 4 的元素数量约为 Test 2 的 $2,003,001/4096 \approx 489$ 倍，但平均时间只从 0.0090 ms 增加到 0.1117 ms，约为 $12.4\times$ 。时间没有按元素数线性增长，原因是小规模测试中 kernel 启动、固定调度开销占比很高；规模增大后 GPU 并行度和内存带宽利用率更充分，单位元素摊销开销下降。

3 Lab 3: Histogram

3.1 Naive Histogram 分析

naive 方案中每个 block 负责一个 bin，并扫描整个输入数组。因此每个输入元素会被读取 num_bins 次。

当 num_bins=256 且 $N = 10M$ 时，总读取次数为

$$256 \times 10,000,000 = 2,560,000,000.$$

若每个输入为 4 bytes，则全局内存读取流量约为

$$2,560,000,000 \times 4 = 10.24 \text{ GB}.$$

该方法低效的原因是重复扫描输入数组，造成全局内存流量按 bin 数放大，并且并行方式没有充分复用每次读取的数据。

3.2 优化后的 Histogram Kernel

```

1  __global__ void kernel(const int* input, int* histogram, int N, int
    num_bins) {
2      __shared__ int sdata[1024];
3
4      int idx = threadIdx.x + blockIdx.x * blockDim.x;
5      int totalthread = blockDim.x * gridDim.x;
6
7      for (int i = threadIdx.x; i < num_bins; i += blockDim.x) {
8          sdata[i] = 0;
9      }

```

```

10     __syncthreads();
11
12     for (int i = idx; i < N; i += totalthread) {
13         atomicAdd(&sdata[input[i]], 1);
14     }
15     __syncthreads();
16
17     for (int i = threadIdx.x; i < num_bins; i += blockDim.x) {
18         atomicAdd(&histogram[i], sdata[i]);
19     }
20 }

```

shared memory 上的 atomicAdd 比 global memory 快, 因为 LDS 延迟更低、带宽更高, 并且只在 block 内竞争。合并阶段每个 block 对每个 bin 至多执行一次全局 atomicAdd。

grid-stride loop for (int i = idx; i < N; i += totalthread) 的作用是让所有线程共同覆盖完整输入, 即使输入规模大于总线程数也能处理, 同时保持较好的负载均衡。

3.3 实测结果与 Speedup

两组测试的实测时间如下:

Test	Serial CPU (ms)	Naive GPU (ms)	Optimized GPU (ms)
Test 2 medium	2.4074	5.5714	0.1857
Test 4 large	23.9073	359.2654	1.7784

据此可得:

$$\begin{aligned}
 \text{Test 2: Optimized vs Serial} &= \frac{2.4074}{0.1857} \approx 12.96\times, & \text{Optimized vs Naive} &= \frac{5.5714}{0.1857} \approx 30.00\times. \\
 \text{Test 4: Optimized vs Serial} &= \frac{23.9073}{1.7784} \approx 13.44\times, & \text{Optimized vs Naive} &= \frac{359.2654}{1.7784} \approx 202.01\times.
 \end{aligned}$$

naive GPU 在这两个测试中都慢于 CPU, 说明重复扫描全局内存的代价很高; 优化版本通过 shared memory 局部统计和一次性合并显著减少了全局内存读取与 global atomic 压力。

3.4 Memory Efficiency

naive 方案读取次数为 $N \times \text{num_bins}$, 优化方案每个元素只从全局内存读取一次, 读取次数为 N 。因此流量降低比例为

$$\frac{N \times \text{num_bins}}{N} = \text{num_bins}.$$

当 num_bins=256 时, 理论读取流量降低 256 $\times$ 。

3.5 Atomic Contention

若所有输入值都相同，例如全为 0，则大量线程会同时更新同一个 bin，atomic contention 最严重，性能会下降。实测中 Uniform 为 0.1863 ms，Gaussian 为 0.1857 ms，而 All-same(0) 为 0.2712 ms，约为 Uniform 的

$$\frac{0.2712}{0.1863} \approx 1.46 \times .$$

若输入值在所有 bin 中均匀分布，更新会分散到多个 bin，contention 较低，因此性能通常更好。

4 Lab 4: Parallel Reduction

4.1 Tree Reduction

对 1024 个元素、block size 为 256 的输入，每个 block 内有 256 个元素参与归约，需要

$$\log_2 256 = 8$$

步 block 内 reduction。

block 数为

$$\left\lceil \frac{1024}{256} \right\rceil = 4,$$

因此 block-level reduction 后剩余 4 个 partial sums。

每一步 reduction 后需要 `__syncthreads()`，因为下一步会读取上一轮其它线程写入的共享内存结果。若没有同步，可能读到尚未完成更新的数据，造成错误结果。

4.2 Atomic 数量分析

当 $N = 1,000,000$ 时：

$$\left\lceil \frac{1,000,000}{64} \right\rceil = 15,625, \quad \left\lceil \frac{1,000,000}{1024} \right\rceil = 977.$$

因此 block size 为 64 时最终全局 atomic 约 15,625 次；block size 为 1024 时约 977 次。

更少的全局 atomic 操作可以降低对同一输出地址的竞争，减少序列化开销，因此可能提升性能。

4.3 算法分析

tree reduction 第一步后，只有一半线程继续参与下一步累加，因此有 1/2 的线程空闲。

warp shuffle 不需要 `__syncthreads()`，因为同一个 warp 或 wavefront 内的线程按 SIMD/SIMT 方式同步执行，shuffle 指令直接在线程 lane 之间交换寄存器值，不经过共享内存。

multi-element per thread 让每个线程先在寄存器中累加多个输入元素，减少 block 数和最终 atomic 次数，同时提高每个线程的顺序内存访问工作量，有利于提升内存带宽利用率。

4.4 实测 Reduction 结果

naive reduction 的测试输出为：

Test	N	Result
Small	10	4
Medium	1,000,000	-74993790
Large	100,000,000	-1984960864

在 Radeon 8060S Graphics 上，对 $N = 1,000,000$ 、block size 256 的 benchmark 结果如下：

Strategy	Time (ms)	Result
Naive Atomic	0.0312	-74993790
Tree (Shared Memory)	0.0250	-74993790
Tree + Warp Shuffle	0.0204	-74993790
Multi-Element + Tree + Shuffle	0.0129	-74993790

四种策略结果一致，验证通过。相对 naive atomic 的 speedup 为：

$$\text{Tree} = 1.25\times, \quad \text{Warp Shuffle} = 1.53\times, \quad \text{Multi-Element} = 2.41\times.$$

这说明 reduction 的性能不只取决于 occupancy。减少 global atomic contention、减少同步开销、让每个线程处理多个元素以改善带宽利用，都会直接提升性能。

5 Lab 5: Monte Carlo Integration

5.1 基础 Kernel 与复杂度

```

1 __global__ void montecarlo(const double* y_samples, double* result,
2                             double a, double b, int n_samples) {
3     int idx = blockDim.x * blockIdx.x + threadIdx.x;

```

```

4
5     if (idx >= n_samples) return;
6
7     double tmp = (b - a) * y_samples[idx] / n_samples;
8     atomicAdd(result, tmp);
9 }

```

每个线程中除以 `n_samples`，可以让每个线程直接产生最终积分中的一项贡献，最后只需要原子加到结果中。等价地，也可以先求和再统一乘以 $(b - a)/n$ ；后者通常与 shared-memory reduction 配合更高效。

`atomic reduction` 的总工作量为 $O(N)$ ，但由于所有线程竞争同一个全局地址，`atomic` 操作会产生严重序列化。

若改为 shared-memory reduction，可以让每个 block 先在共享内存中归约本 block 的局部和，再由每个 block 的一个线程对全局结果执行一次 `atomicAdd`，从而把全局 `atomic` 数量从 N 降到 block 数。

5.2 手工计算

Test 1 中 $a = 0, b = 2, n = 8$ ，样本为

4.1805, 1.7864, 4.7554, 3.9983, 2.2786, 1.1873, 4.1222, 1.442.

样本和为

$4.1805 + 1.7864 + 4.7554 + 3.9983 + 2.2786 + 1.1873 + 4.1222 + 1.442 = 23.7507$.

期望积分结果为

$$\frac{2}{8} \times 23.7507 = 5.937675.$$

程序输出为 5.937675，与手工计算一致。

Test 2 为 $a = b = 1, n = 1$ ，因此积分区间长度为 0，程序输出 0.000000 是正确的。

5.3 Scalability 实测分析

Monte Carlo 积分的端到端运行时间如下：

Test	Sample Count	Real Time (s)
4	100,000	0.104
7	10,000,000	0.968
8	100,000,000	8.284

从 Test 4 到 Test 7, 样本数量增加 $100\times$, 运行时间从 0.104 s 增加到 0.968 s, 约为

$$\frac{0.968}{0.104} \approx 9.31 \times .$$

从 Test 7 到 Test 8, 样本数量增加 $10\times$, 运行时间约增加

$$\frac{8.284}{0.968} \approx 8.56 \times .$$

因此端到端时间不是严格线性。小规模测试受程序启动、文件读取和固定开销影响更大; 规模变大后, 输入读取、host-device 传输、kernel 计算以及 atomic 累加共同决定总时间。

5.4 Atomic Bottleneck

若计算部分需要 1 cycle, atomic 需要 100 cycles, 则每个线程约需 101 cycles, 其中有效计算占比为

$$\frac{1}{1 + 100} \approx 0.99\%.$$

对于 $N = 1M$, 总 cycles 近似为 $1,000,000 \times 101$ 。

shared-memory reduction 可以把大部分累加移动到 block 内部, 只保留每个 block 一次全局 atomic, 从而显著降低全局竞争。

当 block size 为 256 且 $N = 1M$ 时, 全局 atomic 数为

$$\left\lceil \frac{1,000,000}{256} \right\rceil = 3907.$$

5.5 数值精度

double precision 通常提供约 15 到 16 位十进制有效数字。若改用 float, 只有约 6 到 7 位有效数字。对于 $N = 100M$ 的大规模累加, float 更容易出现舍入误差、低位贡献丢失以及不同累加顺序导致的结果差异。

5.6 优化版分析

若优化版本固定使用 256 个 blocks, 则 $N = 1M$ 时只需要 256 次全局 atomic。

grid-stride loop 的作用是让有限数量的线程处理任意规模输入, 每个线程以固定 stride 处理多个样本, 从而保持负载均衡并减少过多 block 带来的开销。

将 $(b - a)/n$ 的乘法放到 thread 0 统一完成, 可以避免每个样本重复执行相同缩放操作, 并让 block 内 reduction 先只处理原始样本和, 减少冗余计算。

6 Lab 6: K-Means Clustering

6.1 Assignment Kernel

assignment kernel 中, 每个线程负责一个点, 并遍历全部 k 个 centroid, 计算最近中心:

```

1  __global__ void find_nearest_centroids_kernel(
2      const float* data_x, const float* data_y, int* labels,
3      const float* centroids_x, const float* centroids_y,
4      int sample_size, int k) {
5      int point_id = blockIdx.x * blockDim.x + threadIdx.x;
6
7      if (point_id < sample_size) {
8          int nearest_centroid_index = 0;
9          float min_distance_sq = INFINITY;
10
11         for (int centroid_index = 0; centroid_index < k; ++
12             centroid_index) {
13             float x = centroids_x[centroid_index] - data_x[point_id];
14             float y = centroids_y[centroid_index] - data_y[point_id];
15             float dist = x * x + y * y;
16
17             if (dist < min_distance_sq) {
18                 min_distance_sq = dist;
19                 nearest_centroid_index = centroid_index;
20             }
21         }
22         labels[point_id] = nearest_centroid_index;
23     }
}

```

当 $k = 100$ 时, 每个线程需要进行 100 次距离计算, 因此单线程复杂度为 $O(k) = O(100)$, 整体 assignment 阶段复杂度为 $O(Nk)$ 。

6.2 Centroid Recalculation 分析

recalculation kernel 先使用 shared memory atomics, 再合并到 global memory, 是为了把大量点的累加先限制在 block 内, 降低 global atomic contention。每个 block 最后只需要按 centroid 数执行合并, 减少对全局内存的竞争。

若某个 centroid 没有任何点分配给它, 代码会保留上一轮 centroid 坐标, 避免除以

0, 并保证空 cluster 不会产生无效值。

6.3 Compute Intensity

对于 $N = 50,000$ 、 $k = 50$ ：

- 每个点的距离计算次数：50。
- 总距离计算次数： $50,000 \times 50 = 2,500,000$ 。
- update 阶段 shared memory atomics：每个点对 x 、 y 和 count 各一次，共 $50,000 \times 3 = 150,000$ 次。
- 若 block size 为 256, block 数为 $\lceil 50,000/256 \rceil = 196$ 。每个 block 对每个 cluster 执行 x 、 y 和 count 三类 global atomic，因此 global atomics 为 $196 \times 50 \times 3 = 29,400$ 次。

Test 4 的实测配置为 $N = 50,000$ 、 $k = 50$ 、max iterations 为 100，端到端运行时间为 0.113 s。每轮迭代的主导计算通常是 assignment 阶段的 $N \times k$ 次距离计算，尤其当 k 较大时更明显。本测试中每轮 assignment 需要 2,500,000 次距离计算，因此它是主要计算来源；update 阶段则主要受 atomic 累加和全局合并影响。

6.4 Memory Access Optimization

centroid 会被所有线程反复读取，可以将 centroid 分批加载到 shared memory 或利用 constant/cache-friendly 访问模式，减少重复 global memory 读取。若 k 较小，也可以提高 centroid 数据的缓存命中率。

当 $k = 1000$ 时，centroid 数组变大，单个线程需要遍历更多中心，缓存工作集增大，cache miss 和内存带宽压力都会上升，同时 assignment 阶段计算量也增加到 $O(1000)$ 每点。

6.5 Convergence

本实现的收敛阈值为

$$dx^2 + dy^2 < 1.0 \times 10^{-8}.$$

当全部 k 个 centroid 都满足该条件时提前终止。

检查收敛比固定运行 max_iterations 更重要，因为许多数据集会提前稳定。提前停止可以减少无意义迭代，节省时间和能耗。

double-buffering 通过交换上一轮 centroid 和新 centroid 的指针，避免在同一数组中边读边写造成数据覆盖，同时减少额外拷贝开销。

7 总结

六个实验的核心思想分别覆盖了一维索引、二维索引与矩阵转置、直方图 atomic 优化、并行归约、Monte Carlo 积分归约以及 K-Means 迭代聚类。总体规律是：GPU 程序不仅要保证线程覆盖完整数据，还需要关注 coalesced memory access、shared memory 局部归约、atomic contention、wavefront 对齐以及 occupancy 与实际瓶颈之间的关系。